

NFC-IET, Multan

Department of Artificial Intelligence

BSAI-2k24

Viva Preparation

10 Most Important Q&A

Cloud Kernel — Cloud Hypervisor Simulation

Submitted by:

Moavia Amir	2k24_BSAI_72
Ali Raza	2k24_BSAI_44
Muhammad Arslan Nasir	2k24_BSAI_26

Semester:	4th Semester
Submitted to:	Mam Amara Nadeem
Subject:	Operating Systems
Document Type:	Viva Q&A; Sheet

CloudKernel — 10 Most Important Q&A;

Q1 What is CloudKernel and what does it simulate?

A CloudKernel is a Java-based simulation of a Cloud Hypervisor — the software that manages Virtual Machines (VMs) on a real cloud server like AWS or Google Cloud. It simulates how an Operating System handles three core problems: system startup coordination, keeping VMs in sync, and sharing limited resources fairly — all using Java threads and synchronization tools.

Q2 What is a CountdownLatch and why did you use it for the boot sequence?

A CountdownLatch is a one-time gate that blocks a thread until a set number of tasks complete. We used it because the system must wait for both Disk and RAM to initialize before starting any VM. It starts at count 2. Each subsystem calls `countDown()` when ready, and the main thread calls `await()` — which unblocks only when the count reaches zero. It is placed in a finally block to prevent deadlock if an exception occurs.

```
bootLatch = new CountdownLatch(2); bootLatch.await(); // Main thread  
waits here bootLatch.countDown(); // Called by Disk and RAM threads
```

Q3 What is a CyclicBarrier and how is it different from CountdownLatch?

A CyclicBarrier is a reusable sync point — all threads must arrive before any can move forward. The key difference is that CyclicBarrier resets automatically after each round, while CountdownLatch can only be used once and then it is done. We used CyclicBarrier for the Global Clock because the clock ticks repeatedly every cycle — not just once at startup.

```
barrier = new CyclicBarrier(3, clockTickAction); barrier.await(); //  
Every VM waits here – auto-resets each round
```

Q4 What is a Semaphore and why did you use it for the Network Port?

A

A Semaphore holds a pool of permits. A thread calls `acquire()` to take one permit and `release()` to return it. We used `Semaphore(2)` because there are only 2 Network Ports but 3 VMs competing for them. This ensures only 2 VMs are on the network at the same time — the third waits until a port is freed. We also passed `true` for fairness so the first VM to wait is the first to get access.

```
networkPorts = new Semaphore(2, true); networkPorts.acquire(); //
Blocks if 0 permits left networkPorts.release(); // Returns permit for
next VM
```

Q5

What is a Thread and how does each VM use it?

A

A Thread is an independent path of execution running inside the same program. Each `VirtualMachine` implements `Runnable` and runs in its own thread — meaning all 3 VMs execute simultaneously, just like real VMs on a cloud server. The `start()` method launches each VM thread. The main thread calls `join()` on each VM thread to wait for all of them to finish before the shutdown phase begins.

```
Thread vmThread = new Thread(new VirtualMachine(...));
vmThread.start(); // VM begins executing vmThread.join(); // Main
thread waits for VM to finish
```

CloudKernel — Q&A; Continued (Q6 – Q10)

Q6 What happens if you remove the CyclicBarrier from the simulation?

A Without the CyclicBarrier, faster VMs would rush ahead to Cycle 2 while slower ones are still on Cycle 1. This is called a race condition — the system goes completely out of sync. In a real hypervisor, this causes data corruption and unpredictable behavior. The barrier enforces that no VM moves to the next cycle until every single VM has completed the current one.

Q7 What is a race condition and does CloudKernel prevent it?

A A race condition happens when two or more threads access a shared resource at the same time and the final result depends on who gets there first — leading to unpredictable, incorrect behavior. Yes, CloudKernel prevents it. The Semaphore(2) ensures only 2 VMs access the Network Port at once. The CyclicBarrier ensures no VM skips ahead in cycles. Both tools eliminate the possibility of threads interfering with each other.

Q8 Why is countDown() placed inside a finally block in BootManager?

A Because the finally block always executes — even if an exception is thrown inside the thread. If we placed countDown() inside the try block and an exception occurred, it would never be called. The main thread would then be stuck at await() waiting forever — this is called a deadlock. Placing countDown() in finally guarantees it always runs no matter what, keeping the system safe.

```
try { Thread.sleep(1500); // Simulate disk work } catch
(InterruptedException e) { Thread.currentThread().interrupt(); }
finally { bootLatch.countDown(); // ALWAYS runs – prevents deadlock }
```

Q9 What is the difference between Semaphore blocking and busy-waiting?

A Busy-waiting means a thread keeps looping and checking repeatedly — like constantly refreshing a page. This wastes CPU time. Semaphore.acquire() is much smarter — it puts the thread to sleep and only wakes it up when a permit becomes available. This is efficient and is exactly how real Operating Systems handle resource waiting without burning unnecessary CPU cycles.

Q1
0

What are .class files and why does the project produce them?

A

.class files are the compiled output of your .java source files. The Java compiler (javac) converts human-readable Java code into bytecode stored in .class files. The JVM (Java Virtual Machine) then reads and executes these files on any operating system. You never edit .class files manually — they are auto-generated every time you compile. In CloudKernel, the -d out flag tells the compiler to place all .class files inside the out/ folder.

```
javac -d out -sourcepath src src/Main.java # Produces: out/Main.class,  
out/core/BootManager.class, etc. java -cp out Main # JVM reads the  
.class files and runs the program
```